

[問題18] (データ抽象化の概念とその実現方法)

次のプログラムは、分数を表現する `Fraction` 型を定義し、2つの分数 `a`, `b` の積を求めるものです。`a`はキーボードより入力し、`b`はプログラム内で定数として与えています。

なお、このプログラムのソースファイルは、`/home/pub/sample/prog18-01.c`にあります。以下の各問に答えなさい。

- (1) コメントに従って空欄を埋め、プログラムを完成させなさい。

(prog18-01.c)

```
#include<stdio.h>

[ (a) ] [ (b) ] {
    int numerator; // 分子
    int denominator; // 分母
} Fraction;

main()
{
    char buff[256];
    [ (c) ] a, b, c;

    // 分数aをキーボードより入力する
    fgets(buff, sizeof(buff), stdin);
    sscanf(buff, "%d/%d", [ (d) ], [ (e) ] );

    // 入力した分数aを表示
    printf("a = %d/%d\n", [ (f) ], [ (g) ] );

    // 分数bを2/3にする
    b.numerator = [ (h) ] ;
    b.denominator = [ (i) ] ;

    // 分数bを表示
    printf("b = %d/%d\n", [ (j) ], [ (k) ] );

    // c = a * b
    c.numerator = [ (l) ] ;
    c.denominator = [ (m) ] ;

    // 乗算結果である分数cの表示
    printf("a * b = %d/%d\n", [ (n) ], [ (o) ] );

    return(0);
}
```

(実行例)

```
$ a.out
3/5
a = 3/5
b = 2/3
a * b = 6/15
```

- (2) このプログラムは分数を表現する `Fraction` 型を定義しているにもかかわらず、そのメリットを生かせていません。なぜですか？問題点は、少なくとも2つあります。

- (3) 次の①～③の関数を使えるものとする、このプログラムはどのように書き換えることができますか？ /home/pub/sample/prog18-03.c の空欄を埋めてみなさい。

① `Fraction getFraction(void);`

機能：キーボードから、分数をn/dの形式で入力する

② `Fraction frcCreate(int numerator, int denominator);`

機能：分子の値numeratorと分母の値denominatorから作られる分数を返す

③ `void frcPrint(Fraction x);`

機能：分数 x をn/dの形で表示する。表示後、改行はしない。

④ `Fraction frcMul(Fraction x, Fraction y);`

機能：二つの分数 x と y の乗算結果 ($x \cdot y$) を返す

- (4) (3)のようにプログラムを書き換えることができたのはなぜですか？関数の外部仕様と内部仕様の役割から説明しなさい。また、書き換えたことによって、main() 関数にはどんな効果が生まれましたか？

- (5) (3)で書き換えたプログラムでは、①～④の関数を利用することで得られたメリットがもう一つあります。どんなメリットか説明しなさい。

- (6) (3)の①～④のように、データ構造に対する操作(処理)を規定する関数を操作関数といいます。また、オブジェクト指向プログラミングでは、操作関数のことをメソッド(method)と呼びます。

①～④の各操作関数(メソッド)を実装し、(3)のプログラムを完成させなさい。

(prog18-06.c)

- (7) prog18-06.c において、Fraction 型のメンバ変数、numerator と denominator を、それぞれ `bunsi`, `bunbo` に変更したいと思います。メンバ変数名の変更後も正しく動作するよう、プログラムを修正しなさい。ただし、①～④以外に新たな関数は作らないものとします。

(prog18-07.c)

- (8) (7)の修正を行った際、どのようなことを問題に感じましたか？また、それを解決するにはどうしたらいいと思いますか？

(9) (7)の修正で生じた問題を解決するために、次の外部仕様で示される2つの操作関数(メソッド)を用意することにしました。

⑤ `int frcGetNumerator(Fraction x);`

機能：分数 `x` の分子の値を返す

⑥ `int frcGetDenominator(Fraction x);`

機能：分数 `x` の分母の値を返す

(a) この2つの関数を実装しなさい。また、⑤、⑥を利用して `prog18-06.c` で使われている操作関数を、できる限り `Fraction` 型のメンバ変数を直接参照しないように書き換えなさい。(prog18-09.c)

(b) `prog18-09.c` では、`Fraction` 型のメンバ変数を直接参照しなければ実装できなかった関数が、⑤、⑥以外にもあったはずですよ。どれですか？関数名を答えなさい

(解 説)

(9)で用意した2つの関数⑤と⑥は、**データ構造内部のメンバ変数を直接参照しなければ実装できない(データ構造の実装に依存する)操作関数**です。

操作関数(メソッド)の中で、データ構造内のメンバ変数を直接参照するものを、アクセサ(**accesser**)といいます。アクセサには、

- ・メンバ変数の値を取り出す … **ゲッター(getter)**
- ・メンバ変数に値を格納する … **セッター(setter)**

の2種類があり、(9)で用意した関数 ⑤、⑥ はゲッターにあたります。

アクセサを用意することで、アクセサ以外の操作関数は基本的にすべてデータ構造に依存しないように実装することができます。このことは、もしデータ構造の表現に変更が生じたとしても、その影響がおよぶのはアクセサのみであり、それ以外の関数は一切修正する必要がないことを意味します。

また、アクセサによってデータ構造内のメンバ変数が隠蔽されることになるため、

- ・予期しないところでメンバ変数の値が書き換えられることを防ぐ
- ・メンバ変数に格納するデータの整合性を検証する
- ・読み出し専用のメンバ変数を定義する

などができるようになります。

(9)の設問(b)で解答した関数は、新たに `Fraction` 型のデータを生成しています。このように、利用したいデータ型のデータを新たに生成する関数を**コンストラクター(constructor)**といいます。

もちろん、コンストラクターも次のようなポインターを用いたセッターを用意することで、データ構造の表現に依存しないように実装することができます。

(Fraction型の分子、分母の値を設定するセッター)

```
int frcSetNumerator(Fraction *x, int numertor);
```

機能：分数 x の分子の値を設定する（戻り値は設定した分子の値）

```
int frcSetDenominator(Fraction *x, int denominator);
```

機能：分数 x の分母の値を設定する（戻り値は設定した分母の値）

しかし、現時点ではまだポインターを学習していないので、このようなセッターを実装できません。そのため、この課題で実装したコンストラクターは、メンバ変数を直接参照することで実装しています。

- (10) 次の文の空欄に適する語句を入れなさい。また、文中の波線部がプログラミングにおいて重要となる理由を説明しなさい。

ここで取り上げた分数データのように、データにはそれぞれ固有の性質があることから、そのデータの性質に合わせた操作をする専用の手続きが存在します。

そこで、データの持つ性質を表現するデータ型とデータに対する様々な操作を一体のものとして定義し、これらをデータを利用するためのインタフェースとして提供する、(a) _____ という考え方が生まれました。(a) _____ により、データの利用方法とデータの内部構造やそれに対する操作の具体的な実現方法が分離されることになります。

(a) _____ の考え方でデータの性質を表現したものを、(b) _____ といいます。C言語などの手続き型言語で(b) _____ を実現するには、まず、データの性質を表す適切な型名と、その型のデータを利用するために必要となる(c) _____ の外部仕様を同時に定義します。この2つ定義が、(b) _____ の外部仕様ということになります。なお、(d) _____ プログラミングでは、(c) _____ のことを(e) _____ と呼びます。

次に、ここで決めた外部仕様を満たすように、(b) _____ の内部仕様となる具体的な(f) _____ と(c) _____ を実装するという手法を用います。この作業はプログラムの規模が大きくなると、外部仕様の設計と内部仕様の実装はそれぞれ異なる人により行われることになるのが一般的です。

(a) _____ (b) _____ (c) _____

(d) _____ (e) _____ (f) _____

(波線部が重要な理由)

- (11) キーボードから入力した2つの分数に対し、四則演算を行った結果を表示するプログラムを作りたいと思います。計算結果は、約分できる場合は既約分数に、整数になる場合は整数で表示することにします(下の実行例2を参照)。

(実行例 1)

```
$ a.out 
a: 2/3 
b: 4/5 
2/3 + 4/5 = 22/15
2/3 - 4/5 = -2/15
2/3 * 4/5 = 8/15
2/3 / 4/5 = 5/6
```

(実行例 2)

```
$ a.out 
a: 2/4 
b: 3/6 
2/4 + 3/6 = 1
2/4 - 3/6 = 0
2/4 * 3/6 = 1/4
2/4 / 3/6 = 1
```

なお、このプログラムでは、設問(9)までで用意した①～⑥に加え、四則演算を行うための操作関数として、以下のものを追加することになります。

- ① Fraction frcAdd(Fraction x, Fraction y);
機能：二つの分数 x と y の加算結果 ($x + y$) を返す
- ② Fraction frcSub(Fraction x, Fraction y);
機能：二つの分数 x と y の減算結果 ($x - y$) を返す
- ③ Fraction frcDiv(Fraction x, Fraction y);
機能：二つの分数 x と y の除算結果 (x / y) を返す
- ④ Fraction frcReduction(Fraction x);
機能：分数 x を既約分数にしたものを返す

ひな形となるサンプルプログラムが /home/pub/sample/prog18-11.c に用意してありますので、①～④の操作関数をすべて実装してこのプログラムを完成させなさい。ただし、データ構造に依存する関数は、最小限にすること。

(prog18-11.c)

(参考) 正数 m と n の最小公倍数は、 $m \times n / (m \text{ と } n \text{ の最大公約数})$ で求めることができます。なお、 m と n の最大公約数は、既に学習したユークリッドの互除法で求められます。

(メモ)

レポート

次のものをE-mailで提出しなさい。

- ・[問題18](2)(4)(5)(8)(9)b および (10)の解答(本文)
- ・[問題18](1)(6)(7)(9)a および (11)の
プログラム(添付ファイル)と実行結果(本文)
- ・演習の感想(本文)

(提出先) E-mail: k-kenta@hamako-ths.ed.jp 件名: プロ技演習18

(提出期限) / () 期限厳守